

30-Day Build Plan — Narri Instagram Store + Admin Dashboard

Solo dev, MERN stack, building from zero.

How to work: vertical slices, not "backend then frontend"

For every feature, do all three steps back to back before moving on:

1. Define the MongoDB schema + API route
2. Build and test the API (Postman/Thunder Client)
3. Immediately wire it to the matching React page/component

This way the site is always in a "working" state, even if incomplete — which matters a lot if you lose a day somewhere, since you're never left with a half-connected mess.

Buffer days

Days 7, 14, 21, and 28 are marked **BUFFER** — no new features, just catch-up on whatever slipped that week. If you lose a day mid-week, don't panic — absorb it into that week's buffer day and adjust the following week's start point. If you lose more than 4 days total across the month, cut a feature (coupons, wishlist, analytics charts) rather than eating into Week 4 testing time.

Week 1 — Foundation + Auth + Product core

Day	Task
1	Repo setup, folder structure, env config. Deploy empty frontend + backend "hello world" to Vercel/Render so the pipeline works from day 1.
2	MongoDB Atlas setup. Design schemas: User, Product, Category, Order, Review, Banner.
3	Auth API: signup, login, JWT, password hashing (bcrypt). Role field (customer/admin).
4	Auth frontend: signup/login pages, protected route wrapper, store token in httpOnly cookie or context.
5	Product API: create/read/update/delete, category linking.
6	Product frontend: homepage product grid, single product page (basic, no reviews yet).
7	BUFFER — catch up on Days 1–6, fix auth edge cases.

Week 2 — Storefront: browsing, cart, checkout

Day	Task
8	Product listing page: filters (category, price) + sort.
9	Search functionality (basic text match on product name).
10	Cart: Zustand/Context store, add/remove/update quantity, persist in localStorage.
11	Cart page UI + mini-cart dropdown.
12	Checkout page: address form, order summary. Order schema + "create order" API.
13	Payment integration (Razorpay/Stripe): test mode, order confirmation on success.
14	BUFFER — catch up, test full purchase flow start to finish.

Week 3 — Admin dashboard

Day	Task
15	Admin dashboard shell: sidebar nav, protected admin-only routes.
16	Admin: Product management UI (add/edit/delete, stock count) connected to existing Product API.
17	Banner API (CRUD + Cloudinary image upload) + Admin banner management UI.
18	Homepage: connect banners to actually display (carousel/hero section).
19	Review API (create/list by product, approve/reject flag) + display reviews on product page.
20	Admin: Review moderation UI (approve/reject/delete).
21	BUFFER — catch up, test banner + review flows end to end.

Week 4 — Orders/sales dashboard, polish, launch

Day	Task
-----	------

- | | |
|----|--|
| 22 | Admin: Orders list view + status update (pending/shipped/delivered). |
| 23 | Admin: Sales summary — total revenue, order count, simple table (skip charts if behind). |
| 24 | Mobile responsiveness pass across storefront (this audience is Instagram traffic — mostly mobile). |
| 25 | Mobile responsiveness pass on admin dashboard + loading/error states everywhere. |
| 26 | Full manual QA: signup → browse → cart → checkout → payment → order shows in admin. Fix bugs found. |
| 27 | Fix remaining bugs from Day 26. Cross-browser check (Chrome/Safari mobile especially). |
| 28 | BUFFER — final catch-up day. |
| 29 | Production deploy, real test transaction with actual payment gateway (not test mode), double-check env vars/secrets. |
| 30 | Final walkthrough with client, handover, buffer for last-minute fixes. |

If you fall behind

Cut in this order (from earlier plan): coupons/discounts → analytics charts (keep raw numbers) → wishlist → advanced image gallery → automated emails (do these manually at launch).